

Software Architecture Document (SAD)

SSS — Shabwa Safety System

المناسبة: تحدي مطوري شبوة 2026 — الموسم الأول: إصدار الوثيقة: 1.0 [cite: 1]
المفكودات [cite: 1]

حالة الوثيقة: الإصدار المعتمد لمعمارية الـ MVP [cite: 1] النطاق الجغرافي: مدينة عتق — محافظة شبوة [cite: 1]

منصة التشغيل: [cite: 1] Responsive Web Application المدة المستهدفة للتنفيذ: 18 يومًا [cite: 1]

1. المقدمة

1.1 الغرض

تحدد هذه الوثيقة المعمارية البرمجية لنظام SSS — Shabwa Safety System، وتوضح كيفية تنظيم مكونات النظام وعلاقاتها وواجهاتها وآليات تخزين البيانات وإدارة المصادقة والصلاحيات والنشر [cite: 1].

تعد هذه الوثيقة المرجع المعماري الأساسي لتنفيذ الـ MVP، وتترجم المتطلبات المحددة في وثيقة SRS إلى بنية تقنية قابلة للتنفيذ والصيانة والتوسع [cite: 1].

1.2 نطاق النظام

SSS هو تطبيق ويب متخصص في إدارة المفكودات والمعتور عليها، يهدف إلى توفير منصة مركزية ومنظمة للإبلاغ عن المفكودات، والإعلان عن الأشياء التي تم العثور عليها، والبحث في التحديثات والسجلات التاريخية [cite: 1].

يعتمد النظام على بنية Modular Monolith في الطبقة الخلفية، ويتكون من [cite: 1]:

- React في الواجهة الأمامية [cite: 1].
- FastAPI في الواجهة الخلفية [cite: 1].
- PostgreSQL في طبقة البيانات [cite: 1].

يتواصل الـ Frontend مع الـ Backend من خلال REST API، ولا يتصل بالبيانات مباشرة [cite: 1].

2. الأهداف والاعتبارات المعمارية

2.1 الأهداف المعمارية

تهدف المعمارية إلى تحقيق [cite: 1]:

1. بساطة التنفيذ خلال مدة المشروع [cite: 1].
2. فصل واضح بين طبقات النظام [cite: 1].
3. سهولة صيانة الكود [cite: 1].
4. أمن مناسب للـ MVP [cite: 1].
5. قابلية التوسع مستقبلاً [cite: 1].
6. اتساق الحالة والبيانات [cite: 1].
7. الحفاظ على السجلات التاريخية [cite: 1].
8. تقليل الاعتماد على خدمات خارجية [cite: 1].
9. توفير بنية قابلة للنشر الفعلي [cite: 1].

2.2 المبادئ المعمارية

تعتمد المعمارية على المبادئ التالية [cite: 1]:

- مبدأ الفصل بين المسؤوليات:** كل طبقة مسؤولة عن نطاق واضح من العمليات [cite: 1].
- مبدأ مصدر الحقيقة الواحد:** PostgreSQL هو المصدر الوحيد للحقيقة لجميع بيانات النظام [cite: 1].
- مبدأ API Boundary:** جميع عمليات الوصول إلى البيانات تمر عبر FastAPI [cite: 1].
- مبدأ أقل تعقيد كافي:** لا تستخدم تقنية أو خدمة إلا إذا كان لها مبرر وظيفي أو معماري [cite: 1].
- مبدأ Modular Monolith:** يتم تقسيم الـ Backend إلى وحدات منطقية مستقلة مع بقائه تطبيقاً واحداً [cite: 1].

3. القيود المعمارية

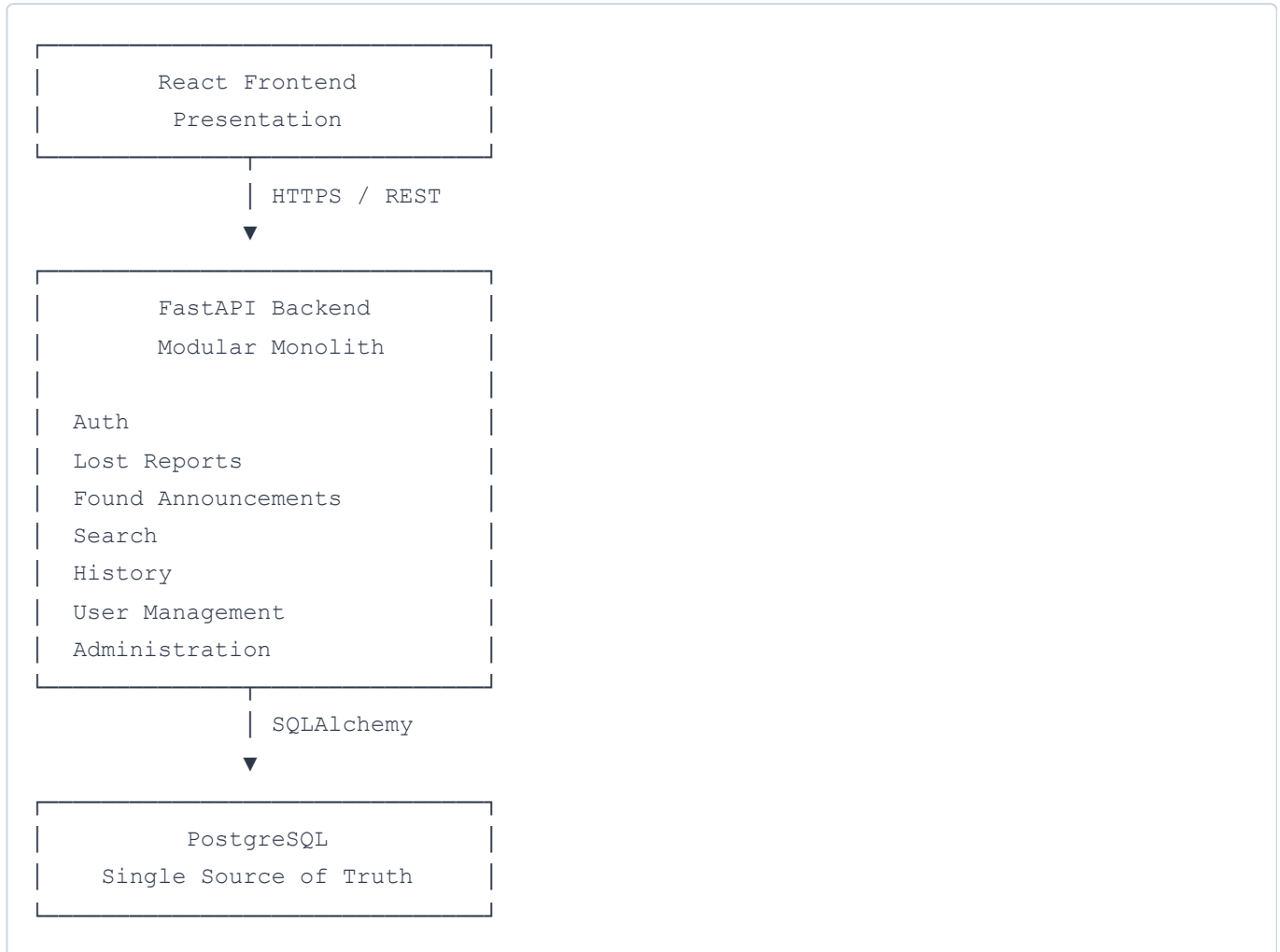
تفرض طبيعة المشروع القيود التالية [cite: 1]:

- مدة التنفيذ 18 يومًا [cite: 1].
- الـ MVP هو الأولوية [cite: 1].
- React هو إطار الواجهة [cite: 1].
- FastAPI هو إطار الـ Backend [cite: 1].
- PostgreSQL هي قاعدة البيانات [cite: 1].
- استخدام REST API [cite: 1].
- استخدام JWT للمصادقة [cite: 1].
- استخدام SQLAlchemy للوصول إلى البيانات [cite: 1].
- استخدام Alembic لإدارة تغييرات مخطط قاعدة البيانات [cite: 1].
- لا توجد صور في الـ MVP [cite: 1].
- لا توجد إشعارات في الـ MVP [cite: 1].
- لا يوجد محرك بحث خارجي [cite: 1].

- لا توجد [cite: 1] Microservices.
- لا يوجد Redis أو WebSockets أو Message Broker [cite: 1].
- لا يتصل الـ Frontend بقاعدة البيانات مباشرة [cite: 1].

4. نظرة معمارية عامة

تعتمد المنظومة على ثلاث طبقات رئيسية [cite: 1]:



5. المعمارية المنطقية

5.1 طبقة العرض — Presentation Layer

تتكون من تطبيق React المسؤول عن [cite: 1]:

- عرض الصفحات [cite: 1].
- إدارة التنقل [cite: 1].
- عرض النماذج [cite: 1].

- التحقق الأولي من المدخلات [cite: 1].
 - إرسال الطلبات إلى API [cite: 1].
 - عرض حالات النجاح والفشل [cite: 1].
 - إدارة حالة الواجهة [cite: 1].
 - التفاعل مع JWT وفق آلية المصادقة المعتمدة [cite: 1].
- لا تحتوي هذه الطبقة على منطق الأعمال الأساسي ولا تتعامل مباشرة مع قاعدة البيانات [cite: 1].

5.2 طبقة API

تتولى FastAPI استقبال طلبات HTTP وتحديد [cite: 1]:

- المسار المطلوب [cite: 1].
- المستخدم المصادق [cite: 1].
- صلاحيات الوصول [cite: 1].
- التحقق من البيانات [cite: 1].
- استدعاء منطق الأعمال [cite: 1].
- إعادة الاستجابة المناسبة [cite: 1].

يتم تنظيم المسارات تحت: `api/v1/[cite: 1]/`

5.3 طبقة الخدمات — Service Layer

تحتوي على منطق الأعمال الخاص بالنظام [cite: 1]. وتشمل مسؤولياتها [cite: 1]:

- إنشاء إبلغات المفقودات [cite: 1].
- إنشاء إعلانات الموجود [cite: 1].
- التحقق من الانتقالات بين الحالات [cite: 1].
- تعديل المحتوى [cite: 1].
- إغلاق المحتوى [cite: 1].
- تسجيل نتائج التسليم [cite: 1].
- تطبيق صلاحيات المستخدم [cite: 1].
- تنفيذ عمليات الإدارة [cite: 1].
- إدارة الانتقال إلى السجل [cite: 1].

لا يجب أن تحتوي طبقة API على منطق الأعمال المعقد مباشرة [cite: 1].

5.4 طبقة الوصول إلى البيانات — Data Access Layer

تتولى التعامل مع PostgreSQL من خلال SQLAlchemy [cite: 1]. وتشمل مسؤولياتها [cite: 1]:

- الاستعلام[cite: 1].
- الإنشاء[cite: 1].
- التعديل[cite: 1].
- قراءة السجلات[cite: 1].
- تنفيذ الفلاتر[cite: 1].
- تنفيذ عمليات البيانات المطلوبة من [cite: 1] Service Layer.

Modular Monolith 6

يبقى FastAPI تطبيقًا واحدًا، لكنه يقسم داخليًا إلى Modules مستقلة منطقيًا[cite: 1].

Authentication Module 6.1

مسؤول عن: إنشاء الحساب، تسجيل الدخول، إصدار JWT، والتحقق من هوية المستخدم[cite: 1].

User Module 6.2

مسؤول عن: بيانات المستخدم، الحسابات، وإدارة بيانات المستخدم المصرح بها[cite: 1].

Lost Reports Module 6.3

مسؤول عن: إنشاء إبلاغ مفقود، قراءة إبلغات المفقودات، تعديل إبلاغ المفقود من مالكه، تغيير الحالة، والانتقال إلى السجل[cite: 1].

دورة حياة إبلاغ المفقود مستقلة عن دورة حياة إعلان الموجود[cite: 1]:

جاري
 — تم العثور عليه
 — إغلاق

Found Announcements Module 6.4

مسؤول عن: إنشاء إعلان موجود، التحقق من اكتمال بيانات الإعلان، النشر، التعديل، الإغلاق، وتسجيل تم التسليم[cite: 1].

دورة حياته[cite: 1]:

جاري
 — تم التسليم
 — إغلاق

Search Module 6.5

مسؤول عن: البحث في التحديثات، البحث في السجلات، تطبيق الفلاتر، واستخدام PostgreSQL للاستعلامات النصية البسيطة والفلاتر المنظمة[cite: 1].

لا يعتمد النظام في الـ MVP على Elasticsearch أو محرك بحث مستقل [cite: 1].

History Module 6.6

مسؤول عن: استرجاع المحتوى الذي خرج من التحديثات، عرض السجلات، تصنيف سبب الخروج، والحفاظ على الفرق بين [cite: 1]:

- إبلاغ مفقود أغلقه الفاقد [cite: 1].
- إبلاغ مفقود تم العثور عليه [cite: 1].
- إعلان موجود تم تسليمه [cite: 1].
- إعلان موجود أغلقه الواجد [cite: 1].

Administration Module 6.7

يوفر وظائف الإدارة للمستخدم ذي الدور الإداري [cite: 1]. يمتلك الـ Admin صلاحيات كاملة، بما فيها [cite: 1]:

- إدارة المستخدمين [cite: 1].
- إدارة المحتوى [cite: 1].
- تغيير حالات السجلات [cite: 1].
- تعديل السجلات [cite: 1].
- إزالة المحتوى من العرض [cite: 1].
- تنفيذ [cite: 1] Soft Delete.
- إدارة الحالات عند الحاجة [cite: 1].

7. المعمارية الصلاحية — Authentication & Authorization

7.1 أنواع المستخدمين

تعتمد النسخة الحالية على مستويين أساسيين [cite: 1]:

- **User**: المستخدم العادي يستطيع إدارة المحتوى الذي أنشأه [cite: 1].
- **Admin**: الـ Admin يمتلك صلاحيات إدارية كاملة [cite: 1].

7.2 إنشاء الحساب

تتكون بيانات الحساب الأساسية من: الاسم، رقم الهاتف أو البريد الإلكتروني، وكلمة المرور [cite: 1]. يجب أن تكون إحدى وسيلتي الاتصال على الأقل متاحة للحساب [cite: 1].

JWT Authentication 7.3

يستخدم النظام JWT للمصادقة [cite: 1].

المسار العام [cite: 1]:

User → Login → FastAPI → Credential Verification → JWT → Protected API Requests

يقوم الـ Backend بالتحقق من الـ JWT في الطلبات التي تتطلب مصادقة [cite: 1].

Authorization 7.4

لا يكفي التحقق من هوية المستخدم؛ يجب أيضًا التحقق من صلاحياته [cite: 1].
مثال [cite: 1]:

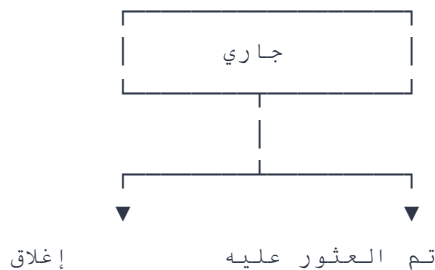
User A → Update Announcement A → ✓
User A → Update Announcement B → ✗

أما الـ Admin فيمتلك صلاحيات تتجاوز قيود الملكية [cite: 1].

8. إدارة إبلغات المفقودات

يرتبط إبلاغ المفقود بحساب المستخدم الذي أنشأه [cite: 1].

8.1 دورة الحياة

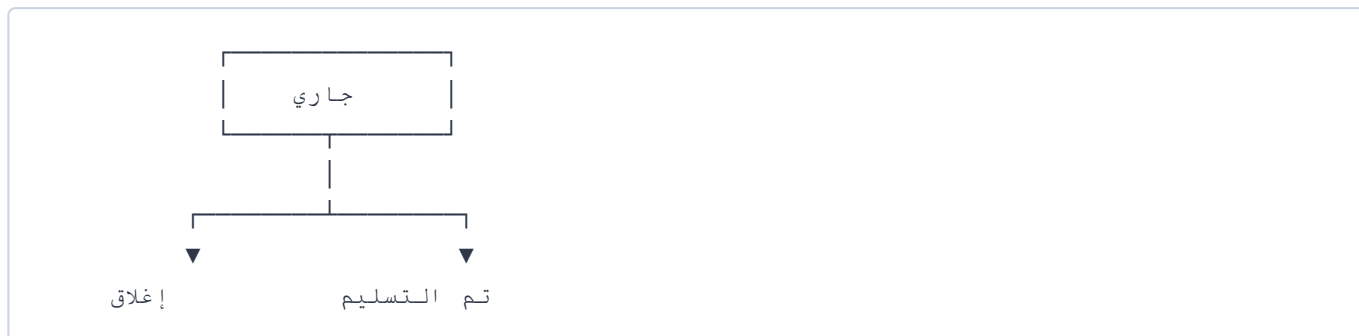


8.2 تم العثور عليه

تعني أن الفاقد سجل أن غرضه لم يعد مفقودًا بالنسبة له [cite: 1]. لا تعني هذه الحالة أن النظام أجرى عملية تسليم، ولا ترتبط هذه الحالة بحالة "تم التسليم" الخاصة بإعلان الموجود [cite: 1].

9. إدارة إعلانات الموجودات

9.1 دورة الحياة



9.2 التحقق قبل النشر

يقتصر التحقق الآلي في الـ MVP على: اكتمال الحقول المطلوبة، صحة أنواع البيانات، صحة القيم، القيود المحددة على طول وحجم المدخلات، وقواعد الإدخال الأساسية [cite: 1].

لا توجد مراجعة بشرية أو Content Moderation متقدمة قبل النشر [cite: 1].

10. قاعدة البيانات

PostgreSQL 10.1

PostgreSQL هو Single Source of Truth لجميع بيانات [cite: 1] SSS. لا توجد في الـ MVP قاعدة بيانات ثانية تحتوي على نسخة مستقلة من البيانات الأساسية [cite: 1].

SQLAlchemy 10.2

يستخدم SQLAlchemy كطبقة ORM للتعامل مع [cite: 1] PostgreSQL. وتتولى طبقة Data Access استخدام النماذج والاستعلامات للوصول إلى البيانات [cite: 1].

Alembic 10.3

يستخدم Alembic لإدارة تغييرات مخطط قاعدة البيانات [cite: 1]. يسمح ذلك بالحفاظ على تسلسل واضح لتغييرات الـ Schema، مثل: [cite: 1] Migration 001, Migration 002, Migration 003...

11. نموذج البيانات المنطقي

الكيانات الأساسية هي [cite: 1]:

- User [cite: 1]
- LostReport [cite: 1]
- FoundAnnouncement [cite: 1]

• HistoricalRecord[cite: 1]

User 11.1

يمثل حساب المستخدم، الخصائص الرئيسية: معرف المستخدم، الاسم، وسيلة الاتصال المستخدمة للحساب، كلمة المرور المجزأة، الدور، وبيانات النظام اللازمة للحساب[cite: 1].

LostReport 11.2

يمثل إبلاغًا عن غرض مفقود، يرتبط بحساب المستخدم الذي أنشأه. يشمل منطقيًا: معرف الإبلاغ، معرف المستخدم، نوع الغرض، الوصف، الموقع، التاريخ، الحالة، والتواريخ التشغيلية[cite: 1].

FoundAnnouncement 11.3

يمثل إعلانًا عن غرض تم العثور عليه. يشمل: معرف الإعلان، معرف الواجد، نوع الغرض، الوصف، موقع العثور، تاريخ العثور، شروط الاستلام، رقم التواصل، الحالة، وبيانات النشر والتعديل[cite: 1].

HistoricalRecord 11.4

يمثل المعلومات الضرورية لفهم خروج المحتوى من التحديثات[cite: 1]. يجب أن يسمح النموذج بتمييز: نوع المحتوى، السجل الأصلي، نتيجة الخروج، ووقت الحدث[cite: 1].

12. تصميم دورة السجلات

لا يتم حذف المحتوى عند انتقاله إلى السجل. بدلًا من ذلك[cite: 1]:

Active Content → State Transition → Historical State

وتبقى البيانات الأصلية قابلة للقراءة ضمن السجلات وفق الصلاحيات المناسبة[cite: 1].

13. Soft Delete

يطبق النظام Soft Delete عند استخدام Admin لإزالة محتوى[cite: 1]. بدل DELETE FROM ... بصورة تؤدي إلى فقدان البيانات، يعتمد النظام على حالة منطقية تشير إلى أن السجل محذوف/مخفي[cite: 1].

ويحقق ذلك: المحافظة على البيانات، إمكانية التتبع، منع فقدان السجل التاريخي، والقدرة على التطوير الإداري مستقبلاً[cite: 1]. ولا يعني Soft Delete أن المحتوى يجب أن يظهر للمستخدمين العاديين[cite: 1].

14. تصميم واجهة REST API

تستخدم جميع واجهات API بادئة الإصدار: /api/v1/[cite: 1]

أمثلة على مجموعات الEndpoints[cite: 1]:

- `api/v1/auth[cite: 1]/`
- `api/v1/users[cite: 1]/`
- `api/v1/lost-reports[cite: 1]/`
- `api/v1/found-announcements[cite: 1]/`
- `api/v1/search[cite: 1]/`
- `api/v1/history[cite: 1]/`
- `api/v1/admin[cite: 1]/`

Authentication API 14.1

تتضمن منطقياً [cite: 1]:

- `POST /api/v1/auth/register[cite: 1]`
- `POST /api/v1/auth/login[cite: 1]`

Lost Reports API 14.2

تتضمن عمليات مثل [cite: 1]:

- `POST /api/v1/lost-reports[cite: 1]`
- `GET /api/v1/lost-reports[cite: 1]`
- `GET /api/v1/lost-reports/{id}[cite: 1]`
- `PATCH /api/v1/lost-reports/{id}[cite: 1]`

Found Announcements API 14.3

تتضمن عمليات مثل [cite: 1]:

- `POST /api/v1/found-announcements[cite: 1]`
- `GET /api/v1/found-announcements[cite: 1]`
- `GET /api/v1/found-announcements/{id}[cite: 1]`
- `PATCH /api/v1/found-announcements/{id}[cite: 1]`

History API 14.4

تتضمن عمليات القراءة والبحث في السجلات التاريخية، مثل [cite: 1]:

- `GET /api/v1/history[cite: 1]`
- `GET /api/v1/history/{id}[cite: 1]`

Admin API 14.5

تتضمن الوظائف الإدارية المتعلقة بـ: المستخدمين، المحتوى، الحالات، Soft Delete، والإدارة العامة [cite: 1]. وتكون جميعها محمية بصلاحيات Admin [cite: 1].

15. التحقق من المدخلات

يحدث التحقق على مستويين [cite: 1]:

15.1 Frontend Validation: يستخدم لتحسين تجربة المستخدم وتقليل الأخطاء قبل إرسال الطلب [cite: 1].

15.2 Backend Validation: يمثل المصدر الموثوق للتحقق، لا يُعتبر اجتياز التحقق في React كافيًا لقبول البيانات، ويجب على FastAPI إعادة التحقق من جميع البيانات القادمة من العميل [cite: 1].

16. التعامل مع الأخطاء

يجب أن يعيد الـ API استجابات متسقة للأخطاء، الحالات الأساسية تشمل [cite: 1]:

- بيانات غير صحيحة [cite: 1].
- مستخدم غير مصادق [cite: 1].
- مستخدم غير مصرح له [cite: 1].
- مورد غير موجود [cite: 1].
- محاولة تعديل مورد غير مملوك [cite: 1].
- انتقال حالة غير مسموح [cite: 1].
- تعارض أو فشل في العملية [cite: 1].

يجب ألا تعتمد الواجهة على نصوص أخطاء داخلية صادرة من قاعدة البيانات [cite: 1].

17. البحث

يعتمد الـ MVP على PostgreSQL في البحث، لا يستخدم النظام: Elasticsearch أو OpenSearch أو أي محرك بحث خارجي [cite: 1].

يمكن استخدام استعلامات "LIKE/ILIKE" للفهرسة النصية البسيطة، مع الفلاتر المنظمة المناسبة [cite: 1]. وتظل PostgreSQL المصدر الوحيد للبيانات [cite: 1].

18. إدارة الحالة

تتم إدارة الحالات في طبقة الأعمال وليس في React فقط [cite: 1].

18.1 LostReport: جاري — (تم العثور عليه / إغلاق) [cite: 1]

18.2 FoundAnnouncement: جاري — (تم التسليم / إغلاق) [cite: 1]

يجب أن يتحقق الـ Backend من صحة أي انتقال قبل تنفيذه [cite: 1].

19. إدارة التحديثات والسجل

التحديثات لا تمثل قاعدة بيانات منفصلة، بل هي عرض Query/Projection للبيانات النشطة [cite: 1]:

PostgreSQL → Active Records → Updates View

وعندما تتغير الحالة [cite: 1]:

Active Record → State Transition → Historical View

وهذا يضمن أن التحديثات والسجلات يعتمدان على نفس مصدر البيانات [cite: 1].

20. إدارة رقم التواصل

رقم التواصل جزء من إعلان الموجود ومحفوظ في قاعدة البيانات مع الإعلان، ولكن الوصول إليه في الواجهة مقيد [cite: 1]:

Visitor → Found Announcement → Contact Number → X
Authenticated User → Found Announcement → Contact Number → ✓

لا يحتاج النظام إلى إنشاء نظام اتصال أو مراسلة داخلي [cite: 1].

21. الأمان

21.1 كلمات المرور: لا تحفظ كلمات المرور بصورتها الأصلية، بل قيم مشتقة باستخدام خوارزمية تجزئة آمنة [cite: 1].

21.2 JWT: يجب التحقق من: صلاحية الرمز، انتهاء الصلاحية، هوية المستخدم، وصلاحيات المستخدم [cite: 1].

21.3 التفويض: يجب تطبيق Authorization على مستوى الـ Backend، ولا يجوز الاعتماد على إخفاء عناصر الواجهة [cite: 1].

21.4 حماية الأسرار: يجب ألا تكون كلمات مرور قاعدة البيانات أو مفاتيح JWT أو أسرار التطبيق مضمنة في المصدر البرمجي أو [cite: 1].Git

21.5 HTTPS: يجب استخدام HTTPS في بيئة النشر الفعلية [cite: 1].

22. Deployment Architecture

يجب أن يدعم النظام بيئتين [cite: 1]:

22.1 Development: Developer Machine (React, FastAPI, PostgreSQL) [cite: 1]

Production / Demo: Internet —► Frontend Hosting —► FastAPI Backend —► PostgreSQL[cite: 1] **22.2**

لا تفرض هذه الوثيقة مزود استضافة محددًا، ويمكن اختيار مزود مناسب لاحقًا [cite: 1].

23. قابلية التوسع

اختيار Modular Monolith لا يمنع التوسع. عند الحاجة مستقبلًا يمكن: إضافة Modules جديدة، تحسين محرك البحث، إضافة الإشعارات، إضافة تخزين الصور، إضافة تكاملات خارجية، إضافة وظائف إدارية، وتوسيع النطاق الجغرافي [cite: 1]. إذا أصبح النظام كبيرًا جدًا، يمكن فصل Modules محددة إلى Services مستقلة، لكن ذلك ليس جزءًا من ال-MVP [cite: 1].

24. القرارات التقنية المؤجلة

لا تحدد هذه الوثيقة مزودًا معينًا للخدمات السحابية، كما لا تفرض: PWA، نظام مراسلة، Notification Service، Object، Storage، Elasticsearch، أو Message Broker [cite: 1]. يمكن دراستها لاحقًا [cite: 1].

25. اعتبارات الجودة

25.1 Usability: مسارات واضحة للبحث، الإبلاغ، إدارة الإعلان، والوصول للسجل [cite: 1].

25.2 Maintainability: فصل الطبقات والModules لتقليل التشابك [cite: 1].

25.3 Reliability: بقاء البيانات عند انتقالها من التحديثات إلى السجل [cite: 1].

25.4 Security: الاعتماد على JWT Authorization والتحقق في ال-Backend [cite: 1].

25.5 Scalability: زيادة قدرات النظام تدريجيًا دون إعادة بنائه [cite: 1].

26. التتبع بين SRS وSAD

القرار المعماري	متطلب SRS
Authentication Module + JWT	حسابات المستخدمين
FastAPI Authorization داخل	صلاحيات الملكية
Lost Reports Module	إدارة إبلاغ المفقود
Found Announcements Module	إدارة إعلان الموجود
PostgreSQL Query/Presentation Layer فوق	التحديثات
History Module	السجل التاريخي
Search Module + PostgreSQL	البحث
Persistent Data + Soft Delete	عدم حذف السجلات
Service Layer + State Validation	التعديل والإغلاق
React	Responsive Web
FastAPI REST API	API
PostgreSQL	البيانات
Alembic	Migrations
SQLAlchemy	ORM

27. المخاطر المعمارية

- 27.1 تضخم نطاق الـMVP: إضافة خدمات غير ضرورية قد تهدد إكمال المشروع في 18 يومًا [cite: 1].
المعالجة: الحفاظ على Modular Monolith وتأجيل الخدمات الإضافية [cite: 1].
- 27.2 تعقيد المصادقة: المصادقة مسؤولية حساسة قد تسبب أخطاء أمنية [cite: 1].
المعالجة: عزل Authentication Module واستخدام JWT والتجزئة المناسبة [cite: 1].
- 27.3 فقدان السجلات: حذف البيانات فعليًا يضر بالسجل التاريخي [cite: 1].
المعالجة: اعتماد Soft Delete وعدم الحذف الفعلي عند النقل للسجل [cite: 1].
- 27.4 تجاوز الواجهة للصلاحيات: إخفاء الأزرار وحده لا يحمي البيانات [cite: 1].
المعالجة: فرض Authorization في FastAPI لكل عملية محمية [cite: 1].

28. الخلاصة المعمارية

تعتمد بنية SSS على Modular Monolith لتحقيق أفضل توازن بين جودة الهندسة وسرعة التنفيذ [cite: 1]. يتكون النظام من [cite: 1]:

React → REST /api/v1 → FastAPI Modular Monolith → SQLAlchemy → PostgreSQL

وتنظم FastAPI إلى Modules مستقلة منطقياً تشمل المصادقة والمستخدمين وإبلاغات المفقودات وإعلانات الموجودات والبحث والسجل والإدارة [cite: 1]. ويستخدم النظام PostgreSQL كمصدر وحيد للحقيقة، مع JWT للمصادقة، وSQLAlchemy للوصول إلى البيانات، وAlembic لإدارة تغييرات قاعدة البيانات [cite: 1].

وتُدار حالات المحتوى داخل طبقة الأعمال بحيث تظل دورة حياة إبلاغ المفقود مستقلة عن دورة حياة إعلان الموجود [cite: 1]. تضمن المعمارية كذلك عدم اتصال الواجهة بقاعدة البيانات مباشرة، وتوفير حدود واضحة بين العرض ومنطق الأعمال والوصول إلى البيانات [cite: 1].

وقد صُممت المعمارية لتكون كافية لتحقيق متطلبات MVP خلال مدة المشروع، وقابلة للتوسع مستقبلاً دون فرض بنية أكثر تعقيداً مما تتطلبه المشكلة [cite: 1].